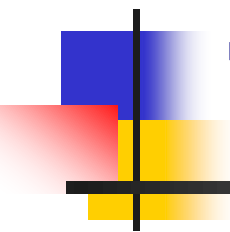
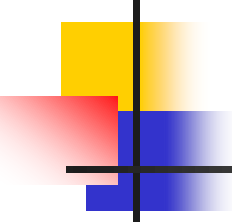


Lab 03

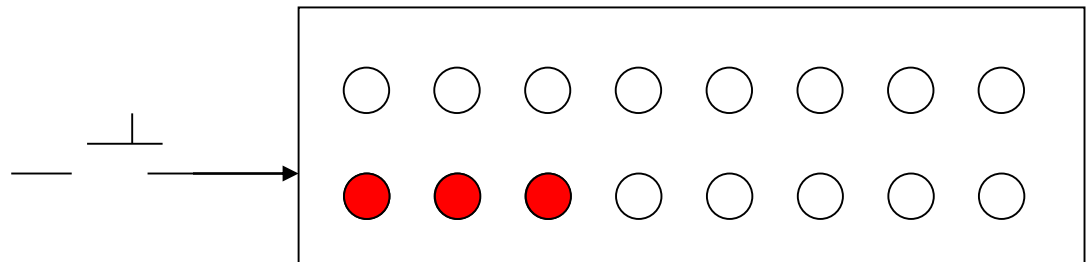


Timer and Interrupt Mechanism

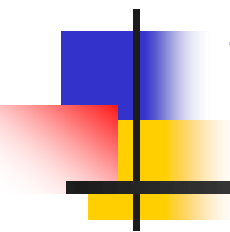


Your Work Today

- Program 8051 to show some LED pattern like last week
- But control using **timer and interrupt mechanism**

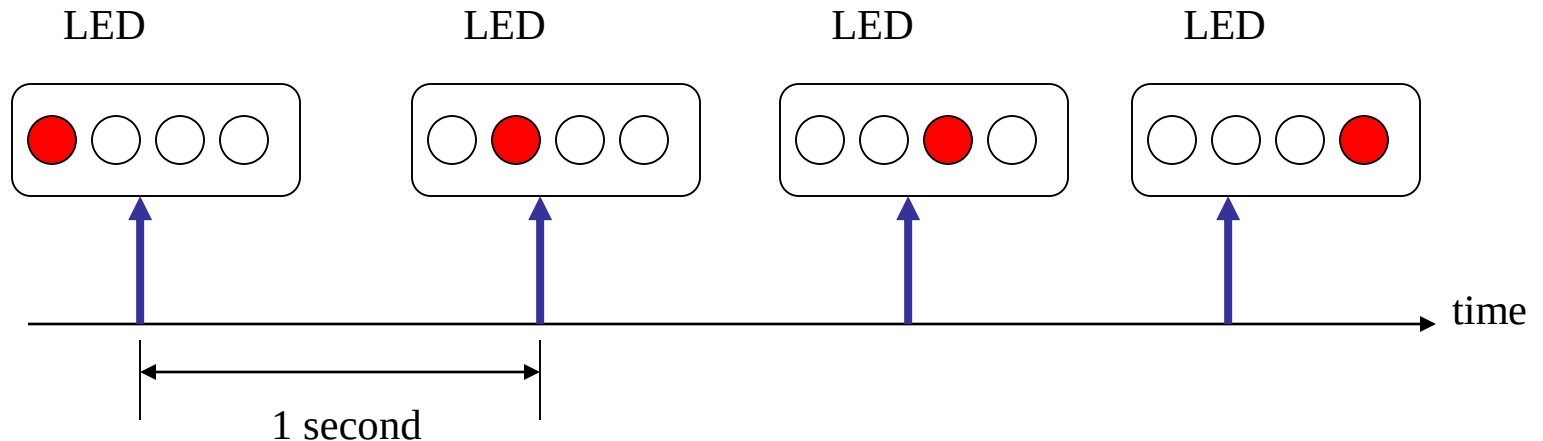


Overview: Program Control Using Timer and Interrupt

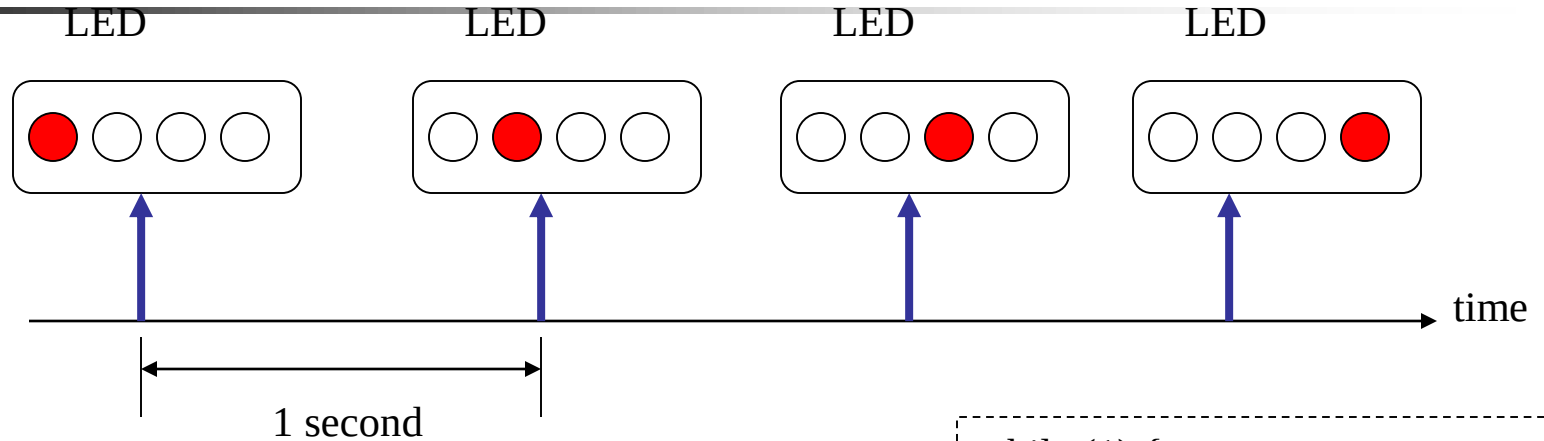


Why Use Timer + Interrupt

- A program to do **precisely** timed control
- Example: make LED switches **precisely** every 1 second



Will You Do It in This Way



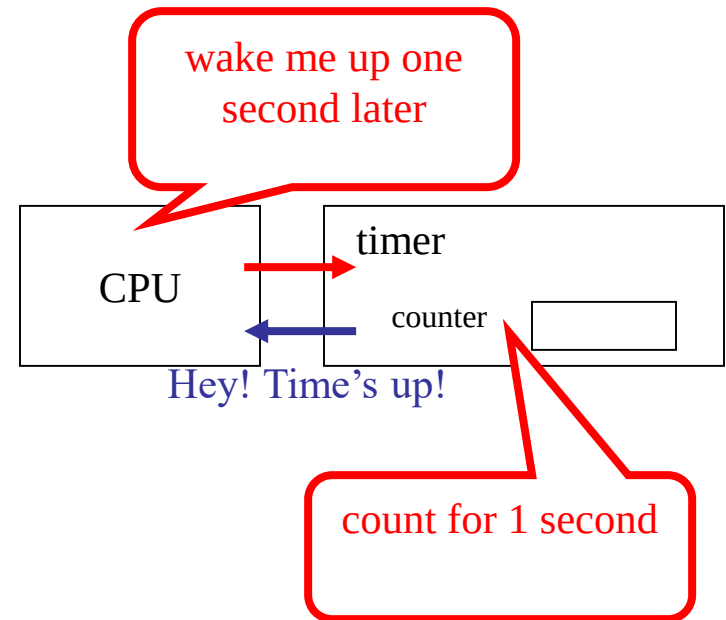
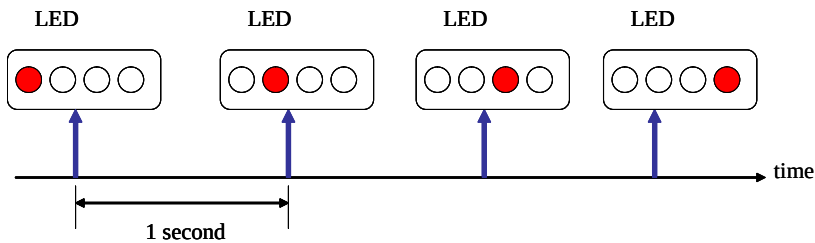
■ How to set N?

- You need a precise cycle count for each assembly instruction

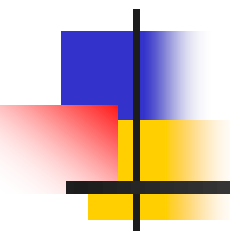
```
while (1) {  
    A = RR(A);    //rotate right  
    P0 = A;  
    delay (N);  
}  
  
delay (int N)  
{  
    int i;  
    for (i=0;i<N;i++);  
}
```

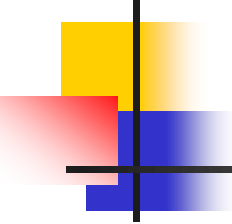
A Better Way for Timed Control

- Use timer + interrupt
- Example:



Basic Concepts of Interrupt Mechanism

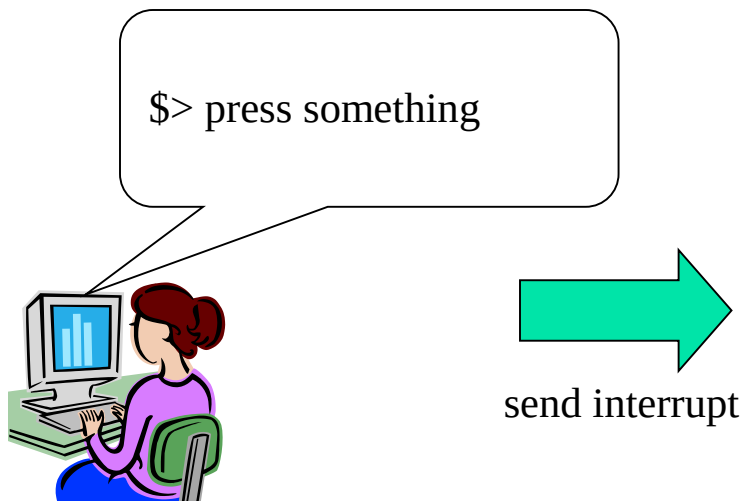




What Is an Interrupt

- To “interrupt” the normal execution of a CPU
 - Turn to do something exceptional and then back to normal execution
 - Usually to serve external I/O devices

Interrupt Normal Execution and then Return



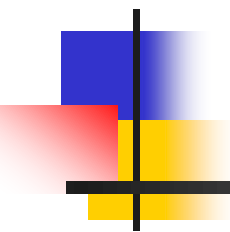
process ID. 1234

```
main()
{
    while (...) {
        ...
        //normal execution
        ...
    }
}

keyboard_intr_handler ()
{
    printf ("A");
}
```

interrupt service routine (ISR)

→ PC

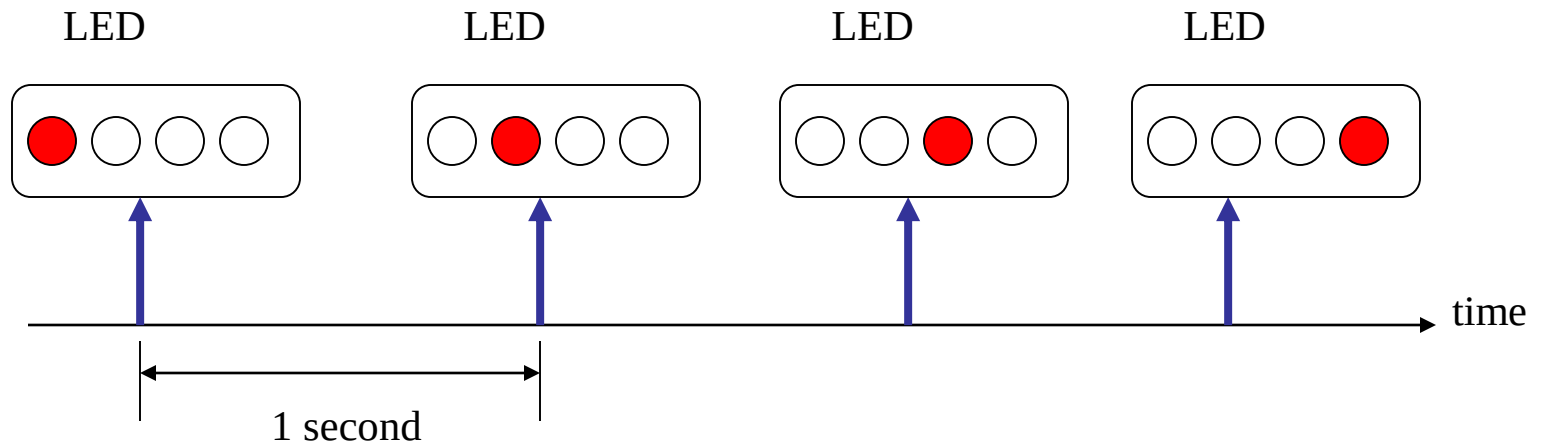


Timer + Interrupt for Timed Control

The conceptual idea

A Better Way for Timed Control

- Use timer + interrupt
- Example:



A Better Way: Using Timer+Interrupt

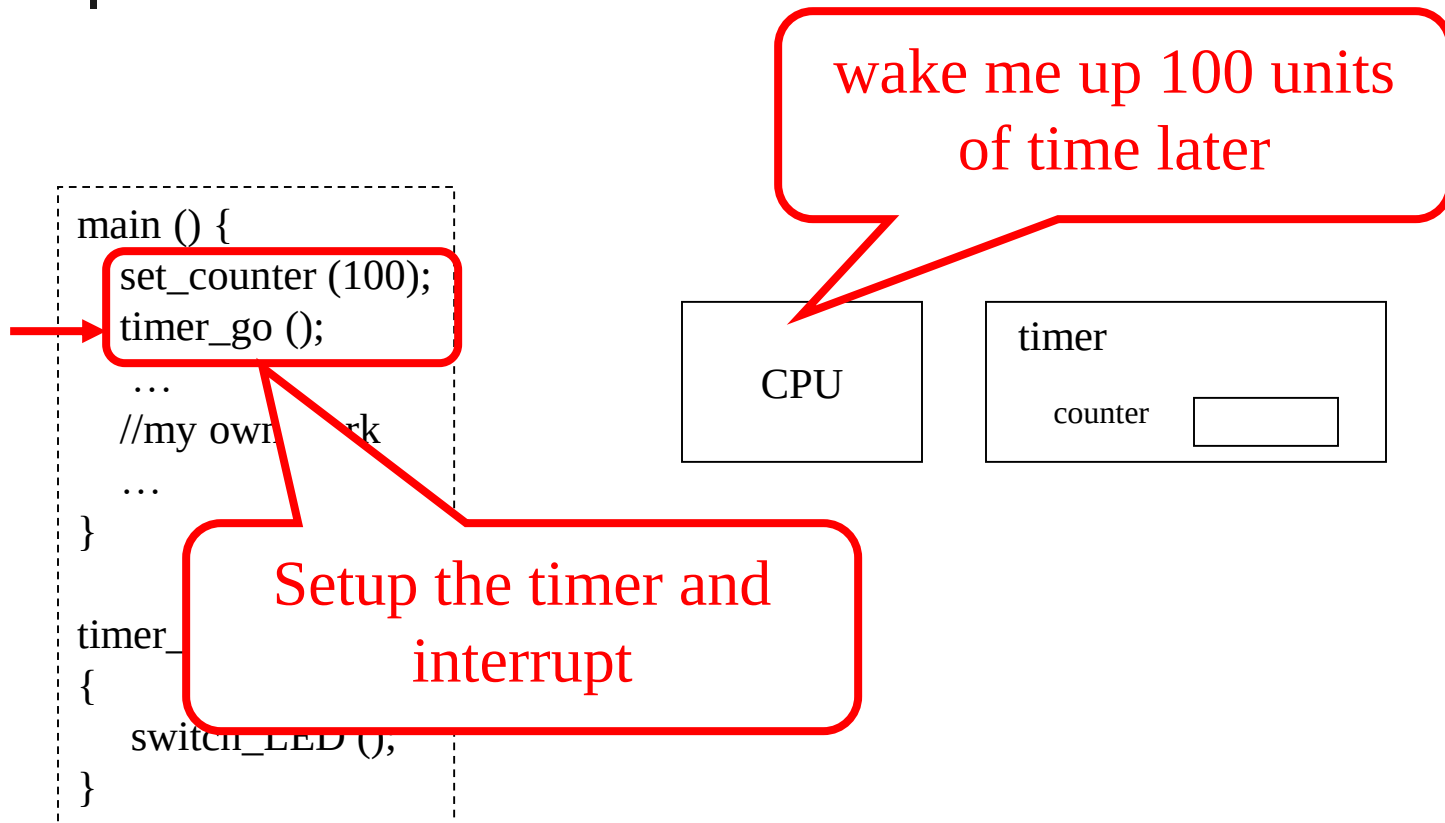
```
main () {  
    set_counter (100);  
    timer_go ();  
    ...  
    //my own work  
    ...  
}  
  
timer_intr_service ()  
{  
    switch_LED ();  
}
```

CPU

timer

counter

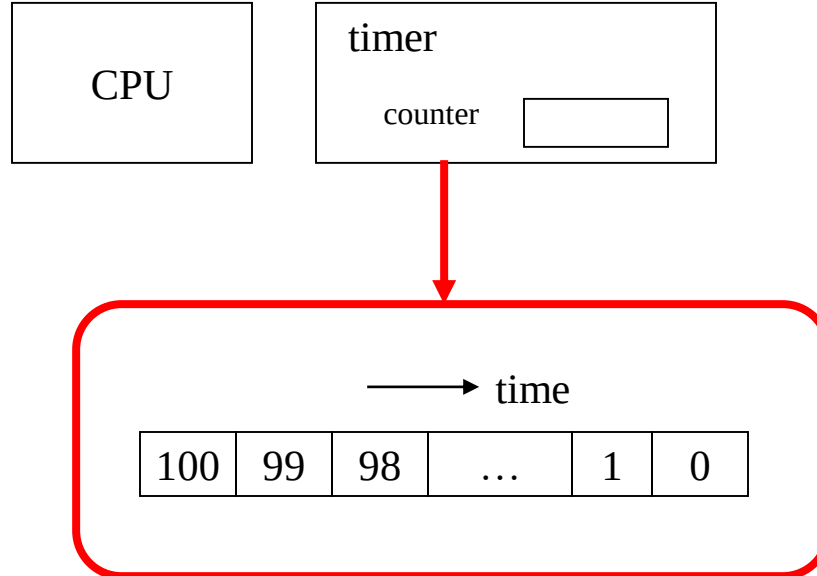
A Better Way: Using Timer+Interrupt



A Better Way: Using Timer+Interrupt

- CPU does its own work and the timer go counting

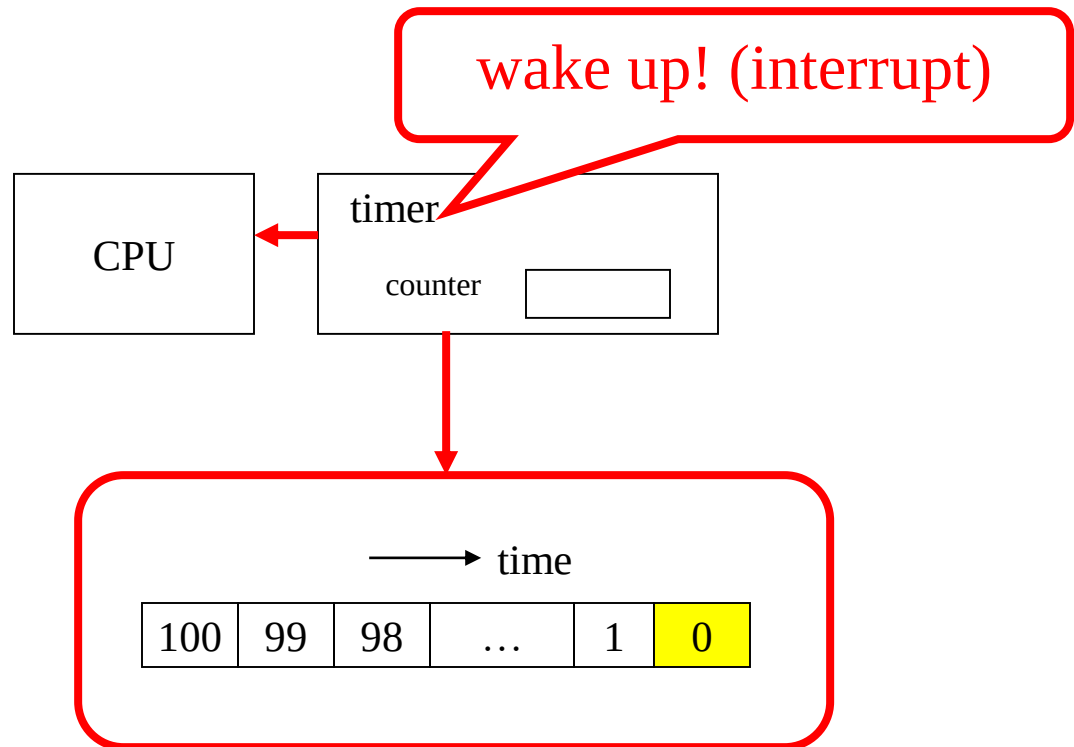
```
main () {  
    set_counter (100);  
    timer_go ();  
    ...  
    //my own work  
    ...  
}  
  
timer_intr_service ()  
{  
    switch_LED ();  
}
```



A Better Way: Using Timer+Interrupt

- The timer sends an **interrupt** to CPU when time-up

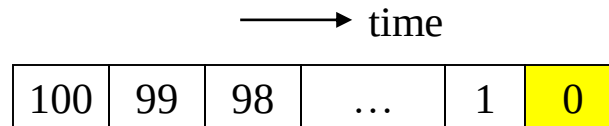
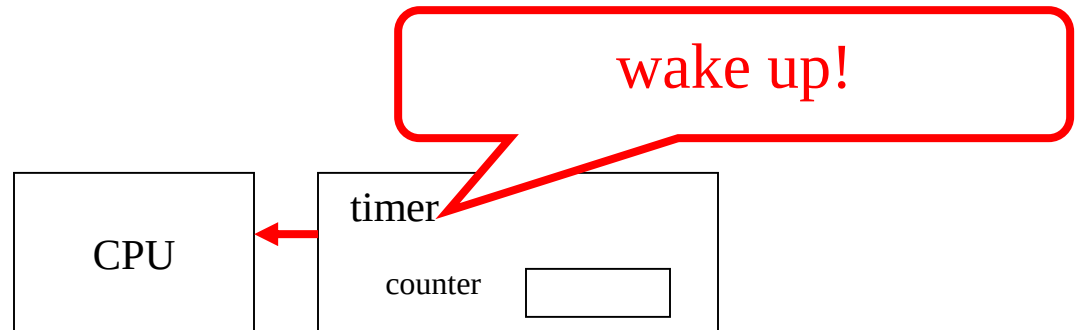
```
main () {  
    set_counter (100);  
    timer_go ();  
    ...  
    //my own work  
    ...  
}  
  
timer_intr_service ()  
{  
    switch_LED ();  
}
```



A Better Way: Using Timer+Interrupt

- CPU turn to the **interrupt service routine**

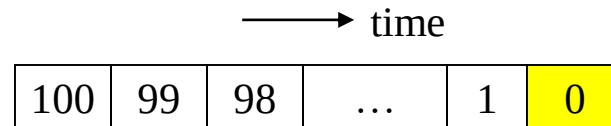
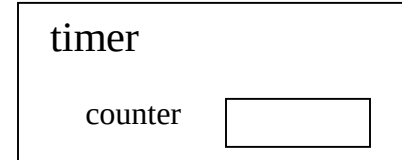
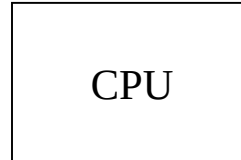
```
main () {  
    set_counter (100);  
    timer_go ();  
    ...  
    //my own work  
    ...  
}  
  
timer_intr_service ()  
{  
→ switch_LED ();  
}
```

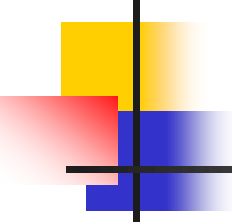


A Better Way: Using Timer+Interrupt

- Then back to its normal execution

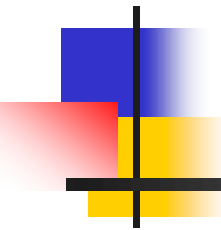
```
main () {  
    set_counter (100);  
    timer_go ();  
    ...  
    //my own work  
    ...  
}  
  
timer_intr_service ()  
{  
    switch_LED ();  
}
```





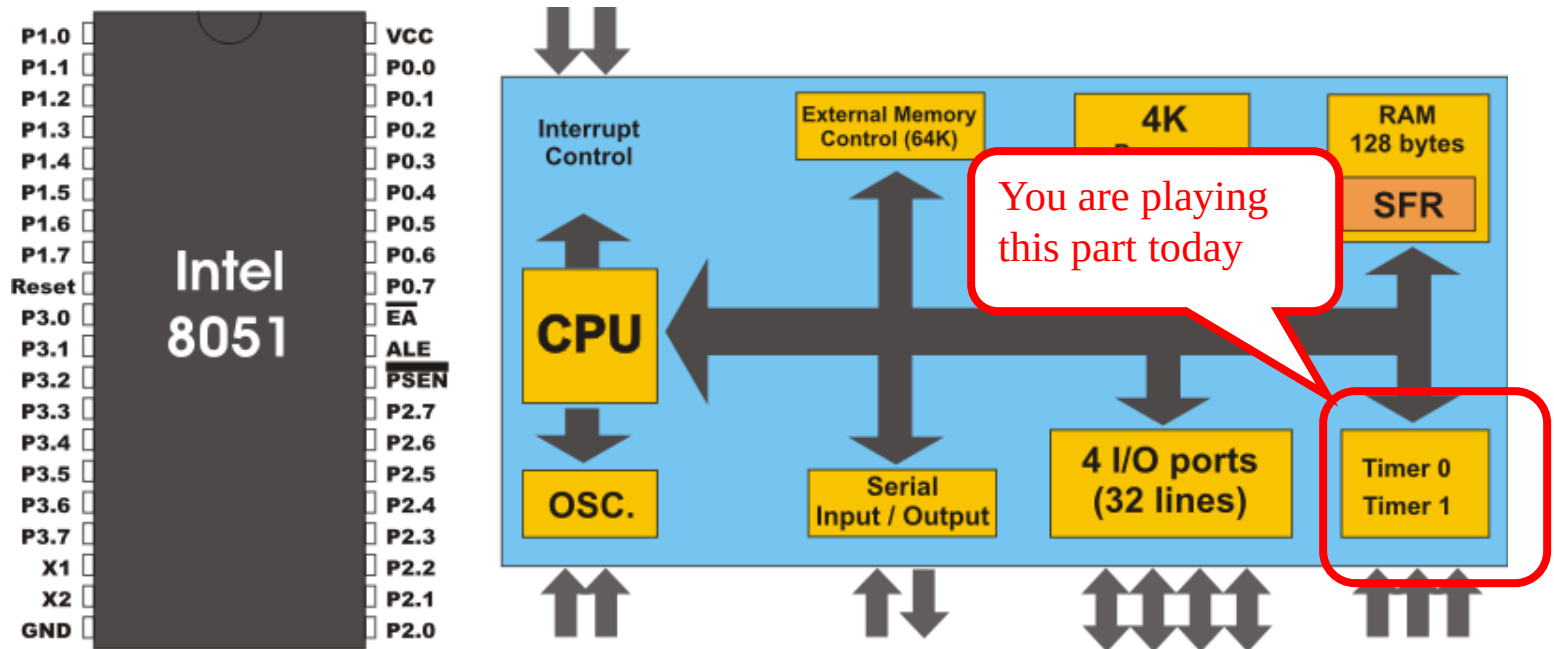
Summary

- Interrupt mechanism:
 - A hardware signal to inform CPU some event has happened
 - Makes CPU change its execution path
 - Turn to “**Interrupt Service Routine**” (ISR)
 - Then return to its normal execution path and status
- Timer:
 - An external counter to count up for specified time
 - Usually inform CPU with interrupt

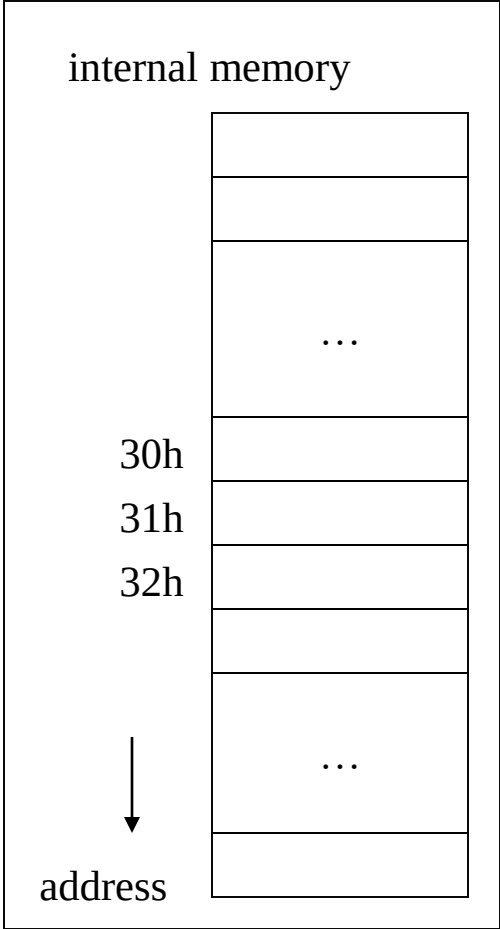
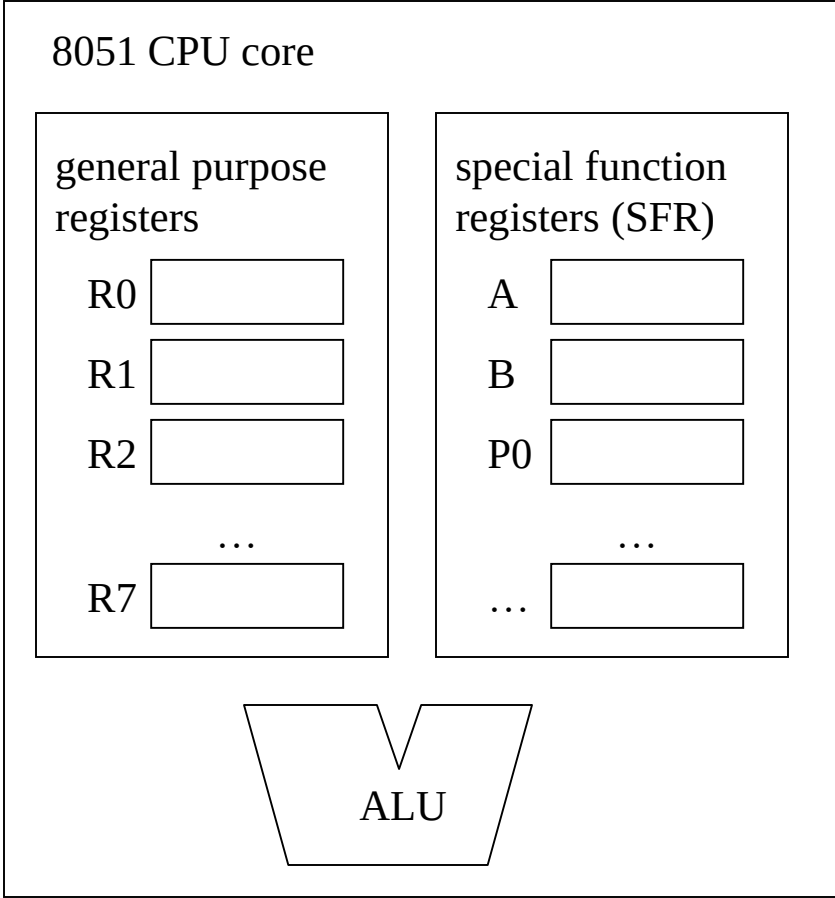


The 8051 part

The 8051 Architecture

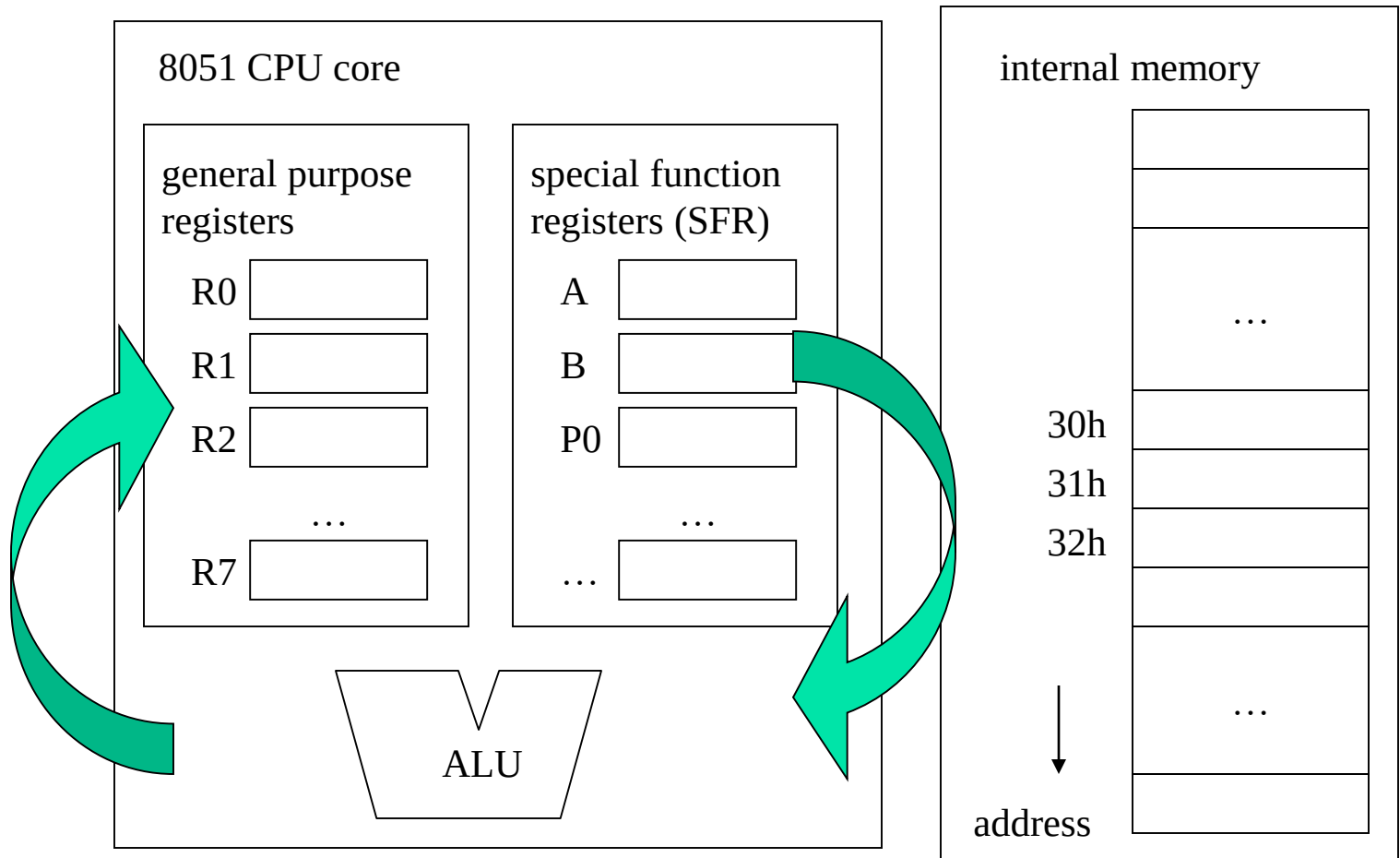


- 



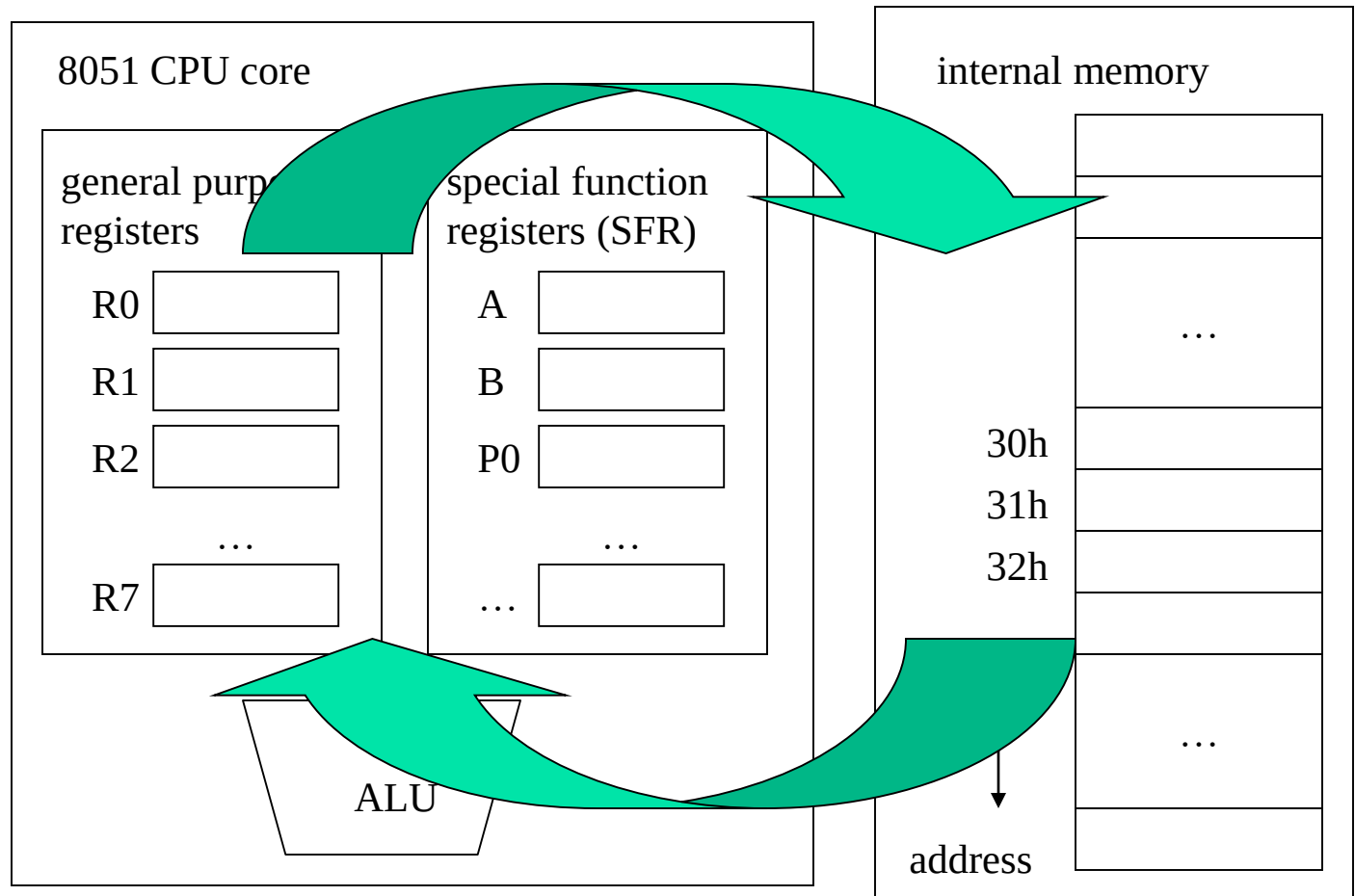
Imagination on 8051 Architecture

- Flow of an arithmetic instruction



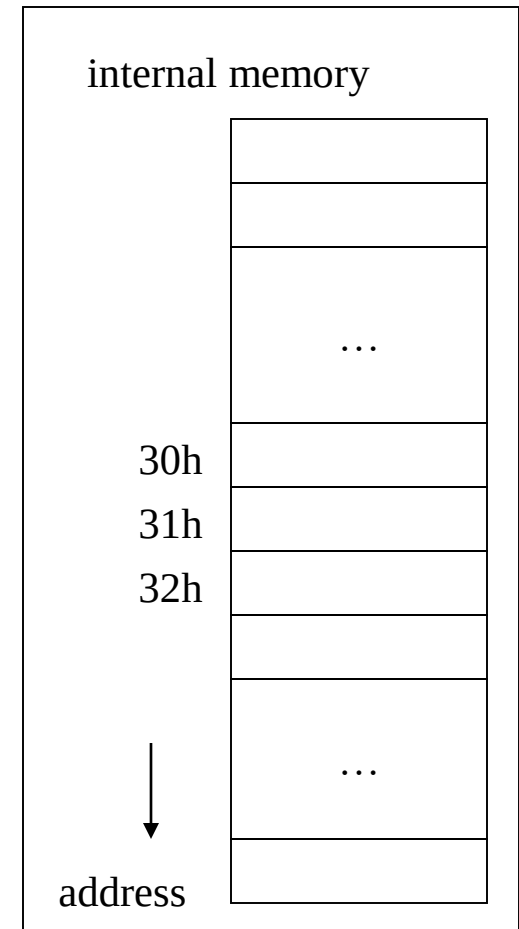
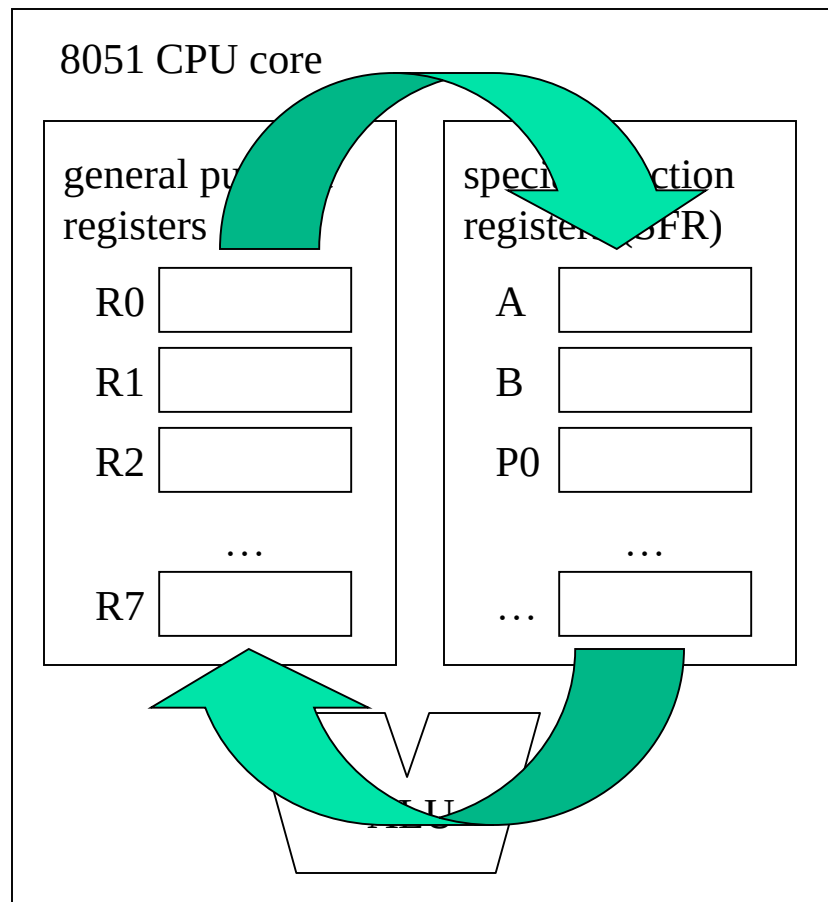
Imagination on 8051 Architecture

- Data movement between memory and registers
- The MOV instruction



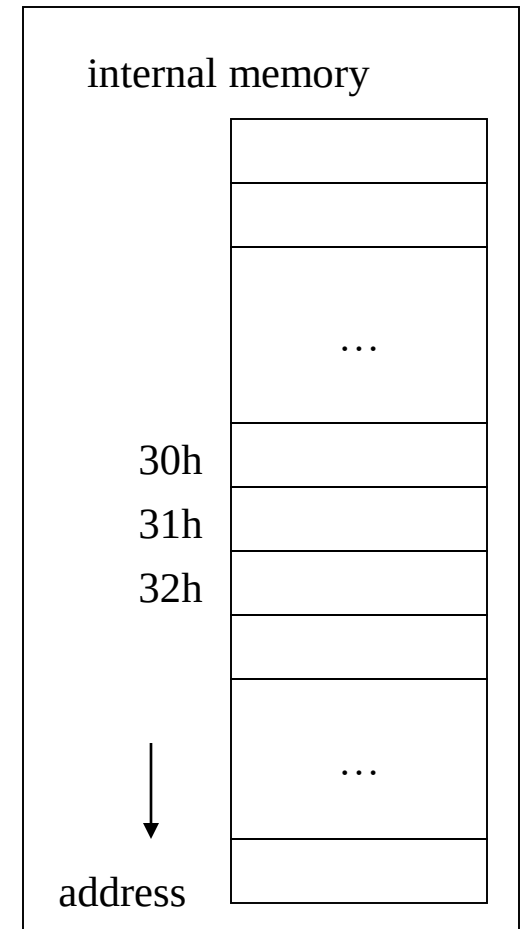
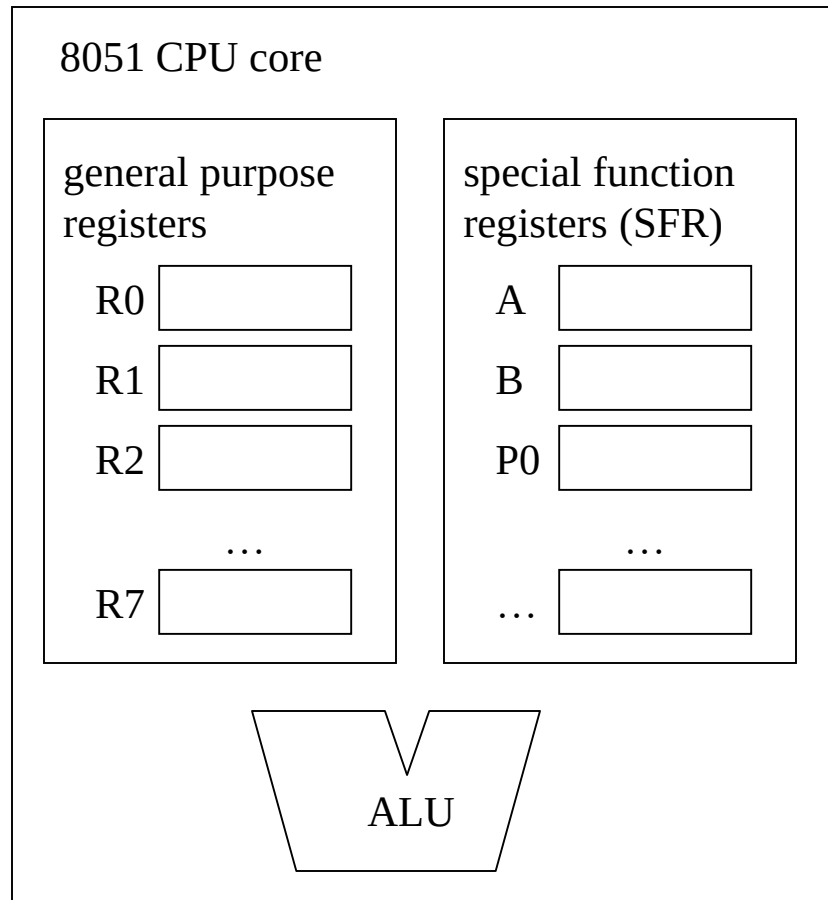
Imagination on 8051 Architecture

- The MOV also for registers

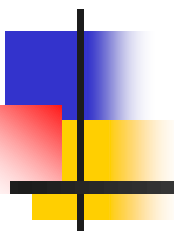


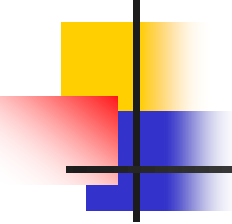
Imagination on 8051 Architecture

- Imagine how data flow in the architecture!



How to Program the Timer and Interrupt Mechanism on 8051



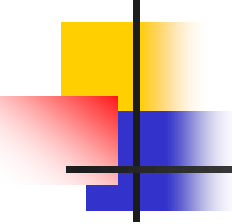


Your Program Looks Like This

```
main ()
{
    TMOD = ???
    TCON = ???
    TH0 = ???
    TL0 = ???
    IE = ???

    while (1); //infinite loop and do nothing
}

Timer_ISR ()
{
    //change LED pattern
}
```



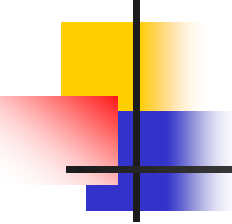
Your Program Looks Like This

```
main ()
{
    TMOD = ???
    TCON = ???
    TH0 = ???
    TL0 = ???
    IE = ???

    while (1); //infinite loop and do nothing
}

Timer_ISR ()
{
    //change LED pattern
}
```

Fill in SFR registers to setup
the timer and the interrupt



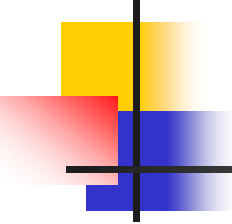
Your Program Looks Like This

```
main ()  
{  
    TMOD = ???  
    TCON = ???  
    TH0 = ???  
    TL0 = ???  
    IE = ???
```

You don't need to branch to control the LED pattern

```
    while (1); //infinite loop and do nothing  
}
```

```
Timer_ISR ()  
{  
    //change LED pattern  
}
```

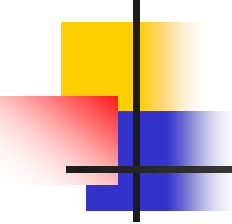


Your Program Looks Like This

```
main ()  
{  
    TMOD = ???  
    TCON = ???  
    TH0 = ???  
    TL0 = ???  
    IE = ???  
  
    while (1); //infinite loop doing nothing  
}
```

The timer interrupt will be executed regularly once setup finished

```
Timer_ISR ()  
{  
    //change LED pattern  
}
```



Things You Need to Know

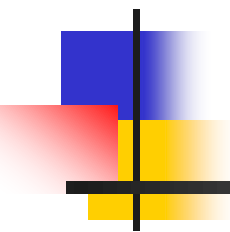
```
main ()  
{  
    TMOD = ???  
    TCON = ???  
    TH0 = ???  
    TL0 = ???  
    IE = ???  
  
    while (1); //infinite loop and do nothing  
}
```

How to setup SFR registers for the timer and the interrupt?

```
Timer_ISR ()  
{  
    //change LED pattern  
}
```

Where to place the timer ISR?

How to Program 8051's Interrupt Mechanism



Things You Need to Know

- (1) How to set SFR registers
- (2) Where to place interrupt service routine (ISR)?

F8									FF
F0	B								F7
E8									EF
E0	ACC								E7
D8									DF
D0	PSW								D7
C8									CF
C0									C7
B8	IP								BF
B0	P3								B7
A8	IE								AF
A0	P2								A7
98	SCON	SBUF							97
90	P1								8F
88	TCON	TMOD	TL0	TL1	TH0	TH1			87
80	P0	SP	DPL	DPH				PCON	

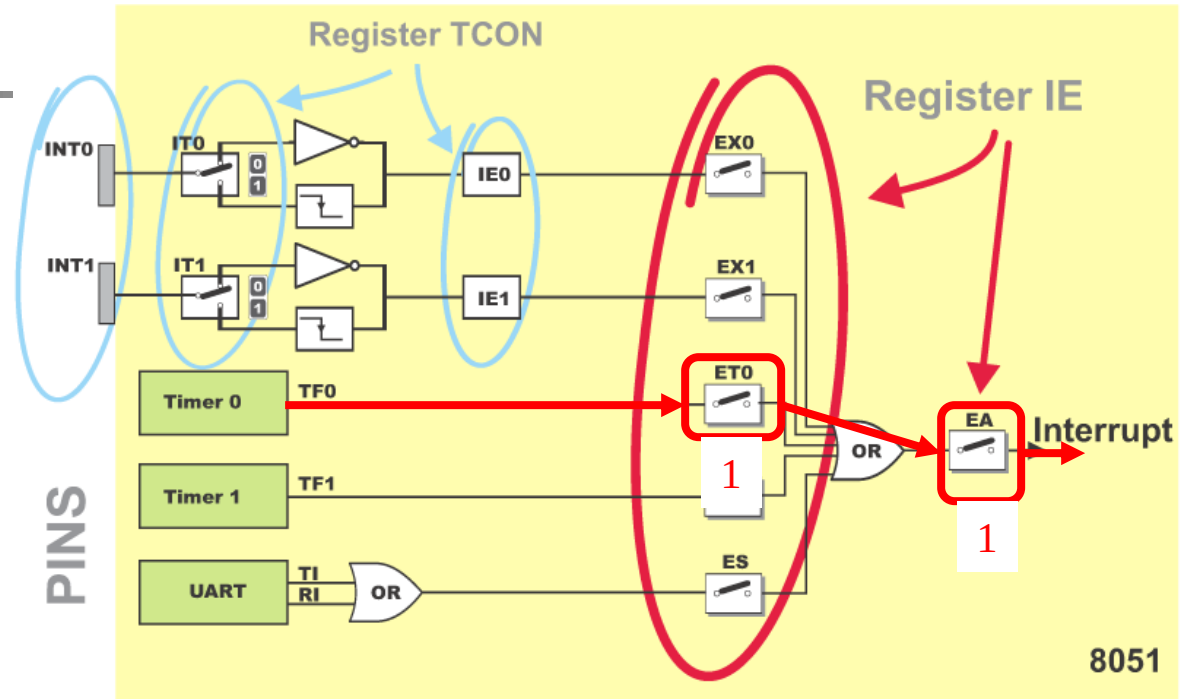
↑ Bit-addressable Registers

interrupt enable

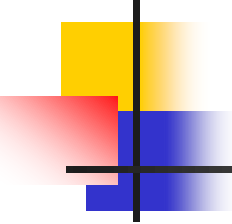
registers to control the timer

The IE Register

	1	X	0	0	0	0	1	0	Value after Reset
IE	EA		ET2	ES	ET1	EX1	ET0	EX0	Bit name
	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	

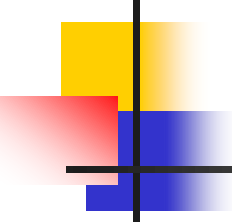


- Imagine a path in the figure
- Set switches in the figure to enable the path



Where's the Interrupt Service Touting

- The interrupt source vs. starting address of the ISRs:
 - IE0: 0x3 (external interrupt)
 - TF0: 0xb (timer 0 overflow)
 - TF1: 0x1b (timer 1 overflow)
 - RI, TI: 0x23 (for UART)



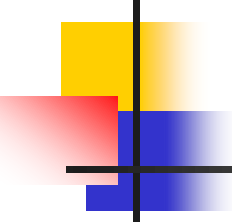
Your Program Will look like

```
org 0H
AJMP MAIN
org 0Bh
AJMP show_LED
...
```

assembler directive: place my code
from address 0x0b

```
MAIN:
...//your main program
```

```
show_LED: //the timer interrupt service routine
MOV R0, #LED_pattern
MOV P0, R0
...
```



Your Program Will look like

```
org 0H  
AJMP MAIN  
org 0Bh
```

timer 0 ISR starts from here

```
AJMP show_LED
```

```
...
```

```
MAIN:
```

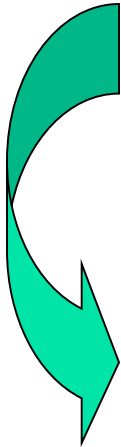
```
...//your main program
```

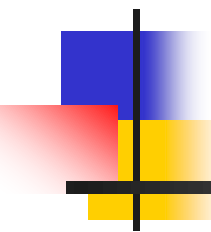
```
show_LED: //the timer interrupt service routine
```

```
MOV R0, #LED_pattern
```

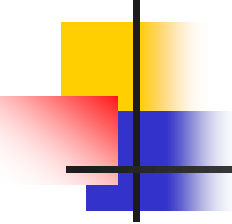
```
MOV P0, R0
```

```
...
```





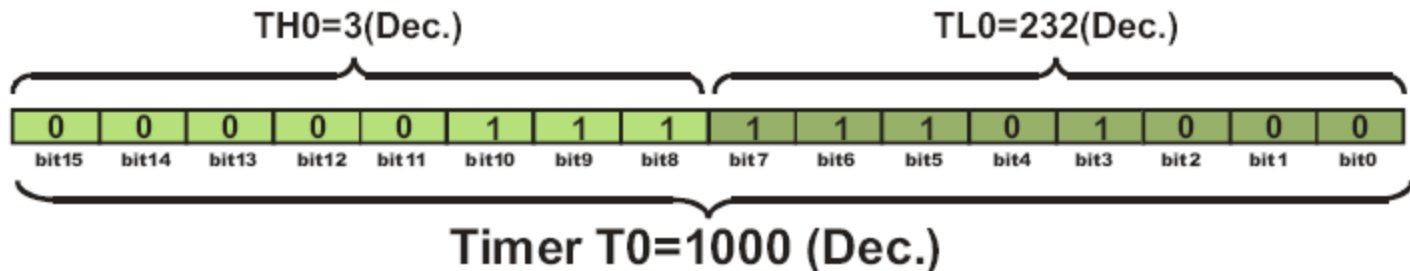
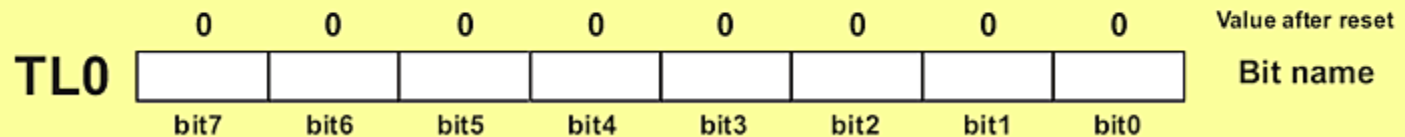
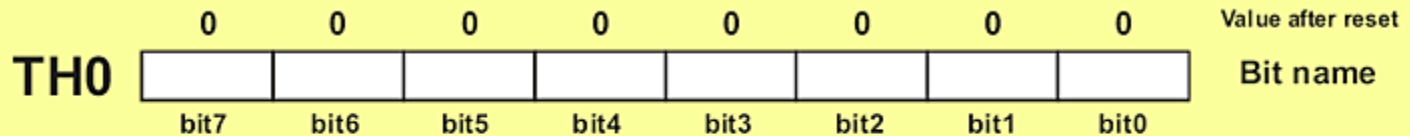
How to Program 8051 Timer



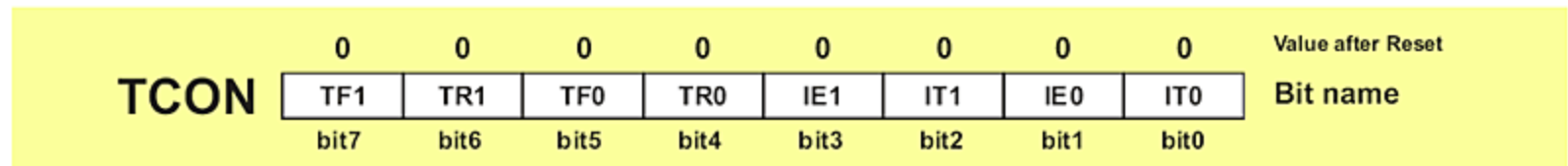
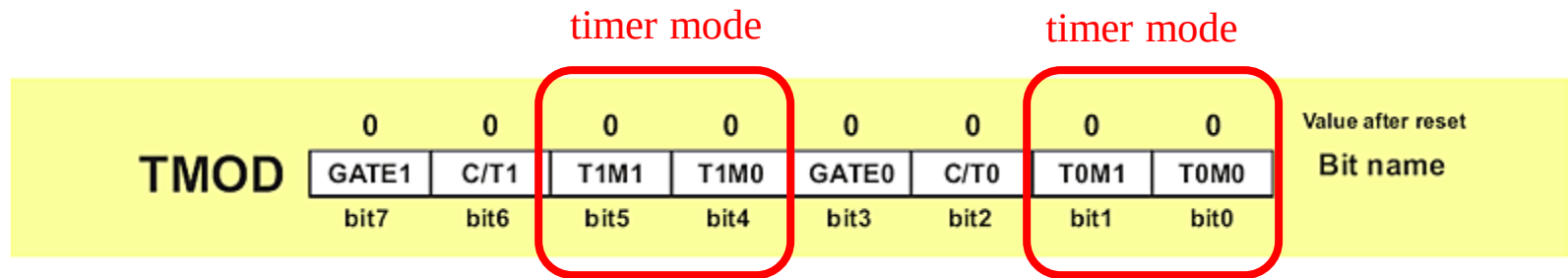
Overview of 8051 Timer

- Two timers:
 - timer 0: {TH0, TL0}
 - timer 1: {TH1, TL1}
- Four modes (set by TMOD register)
 - 0: 13-bit mode
 - 1: 16-bit mode
 - 2: auto reload mode
 - 3: split mode

The SFRs for Counting



Registers to Control the Timer Mode

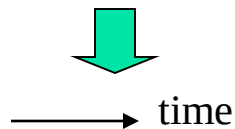
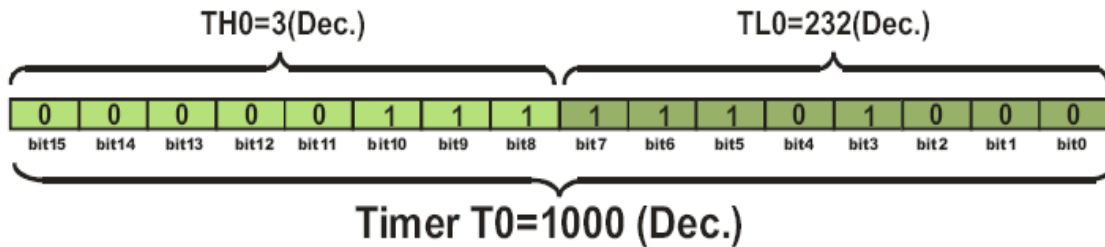


How 8051 Timer Works

Step 1: set {TH, TL}=N

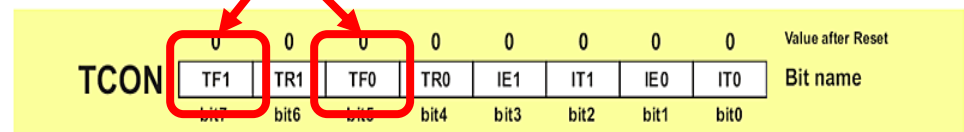
Step 2: enable counting by setup TMOD, TCON

Step 3: wait for timer overflow (check TCON)



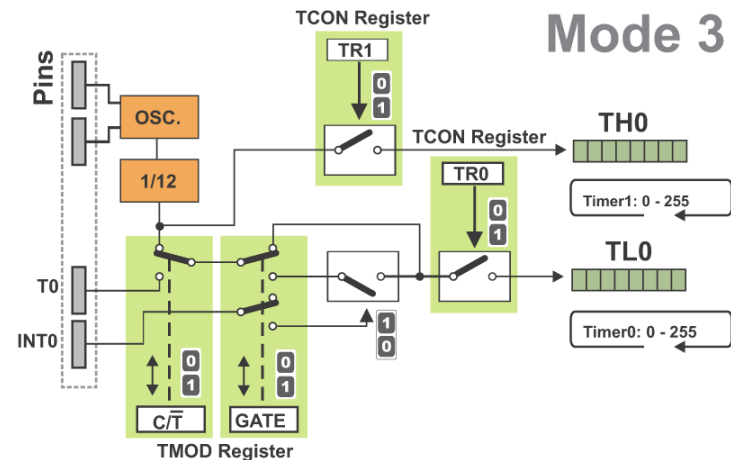
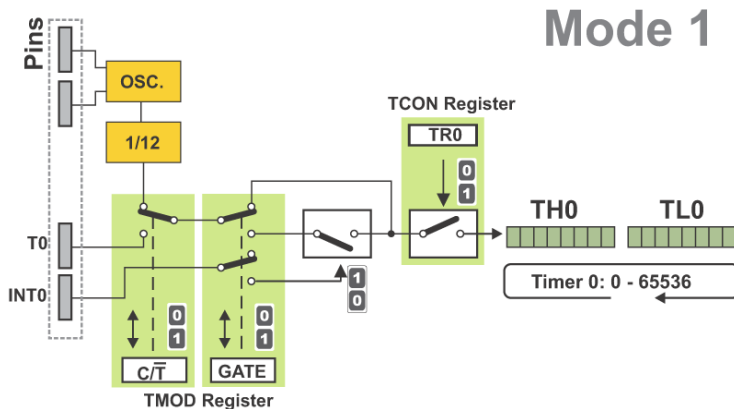
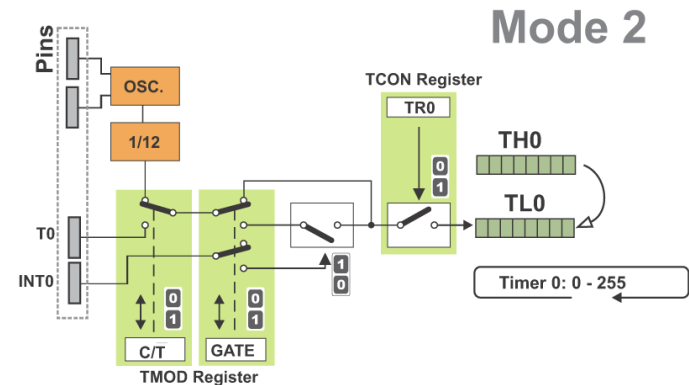
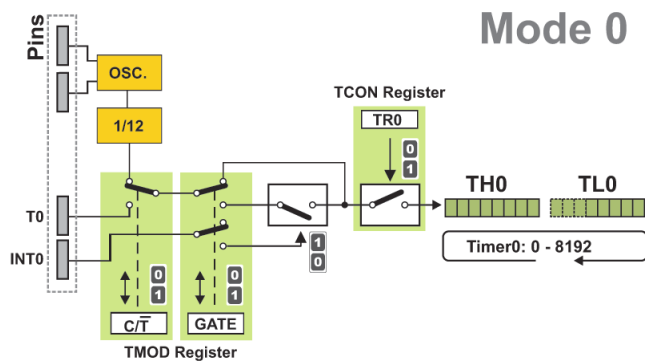
1000	1001	1002	...	0xffff	0
------	------	------	-----	--------	---

TF=1 to indicate timer overflow (0xffff reached)



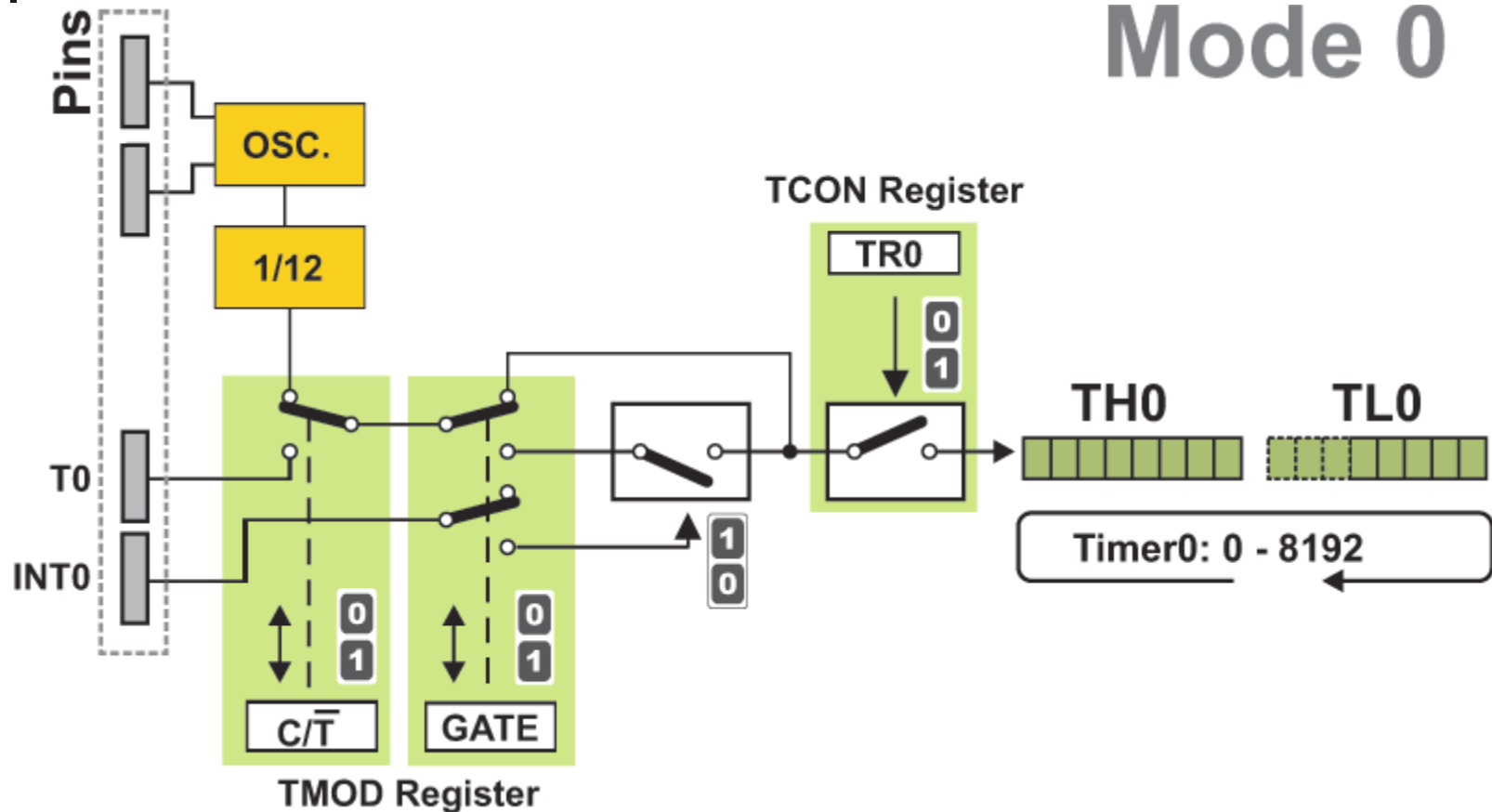
How to Set Lots of Bits in TMOD, TCON, and IE

Check these figures for the four timer modes

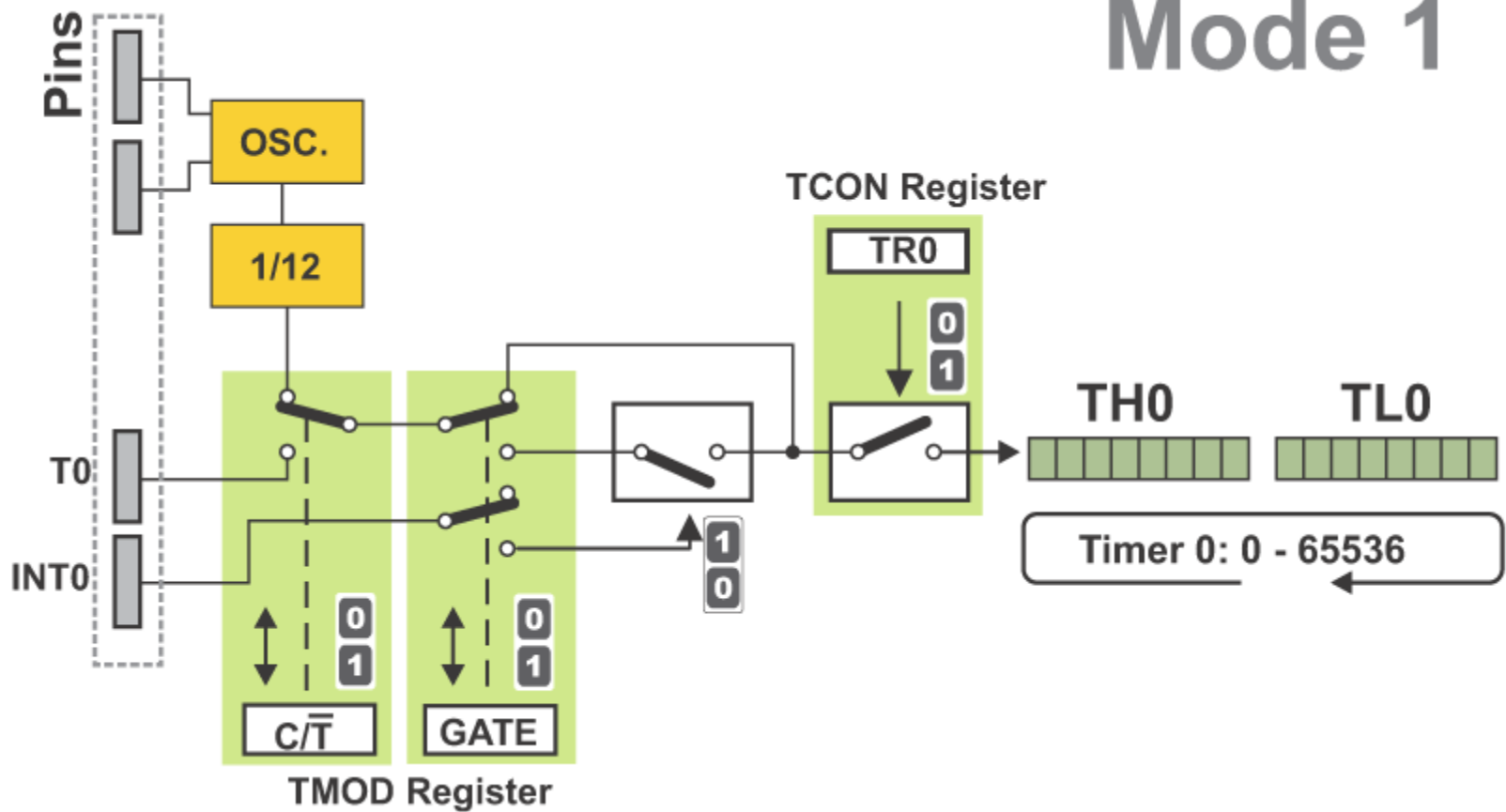


Timer Mode 0

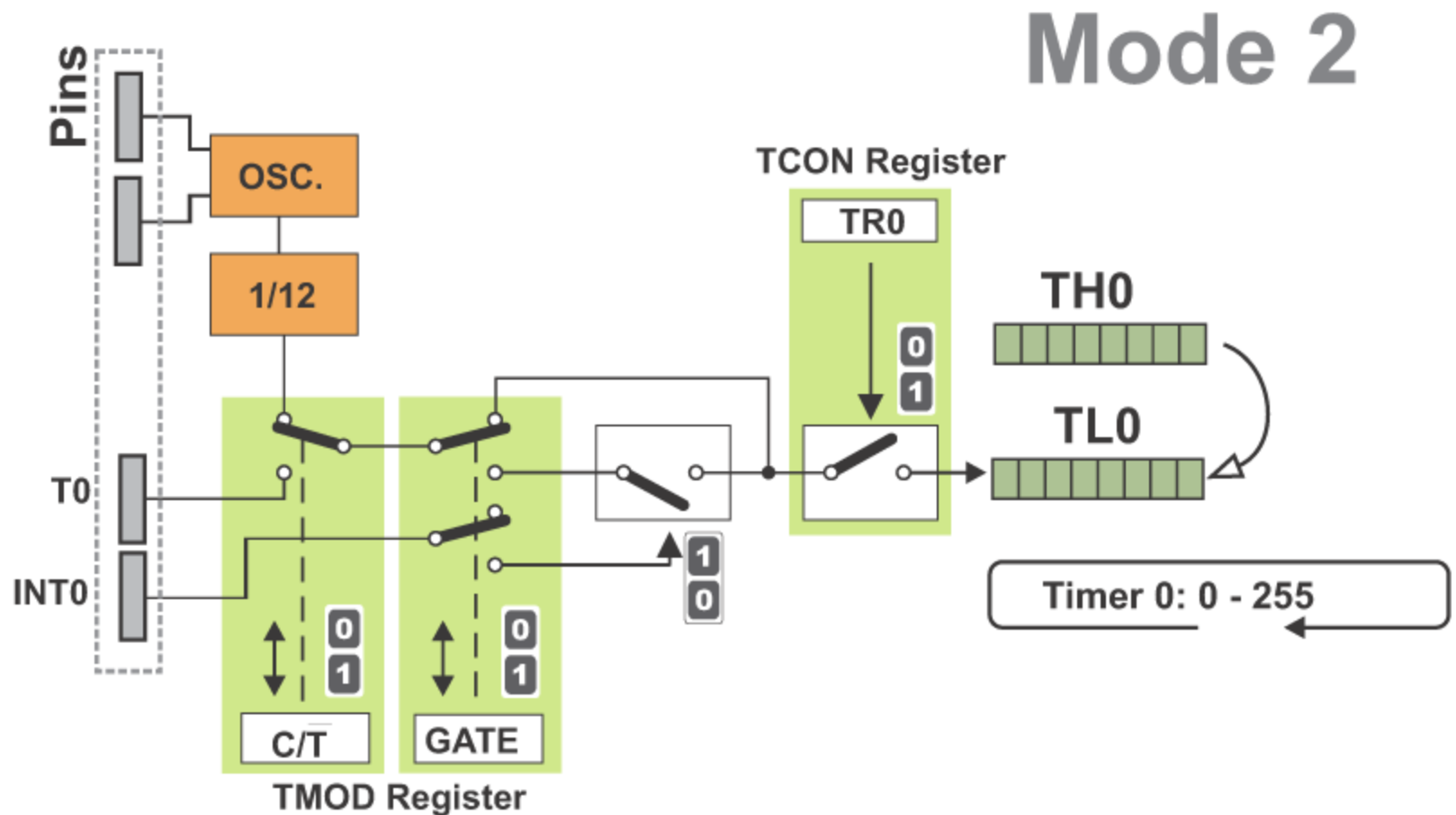
Mode 0



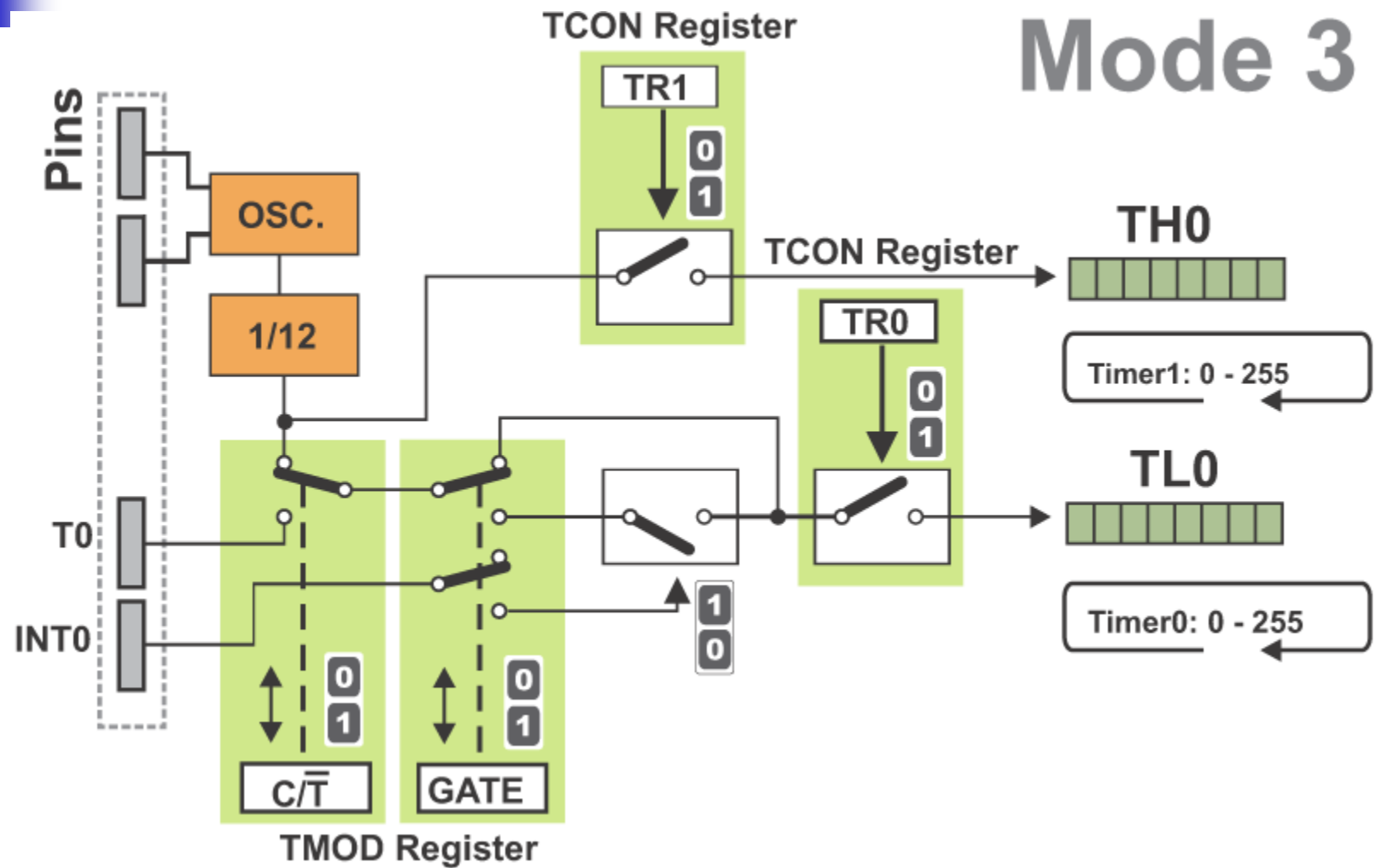
Timer Mode 1



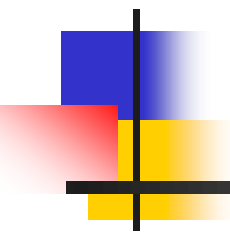
Timer Mode 2



Timer Mode 3

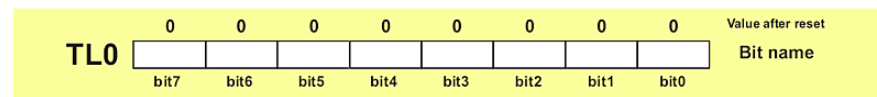
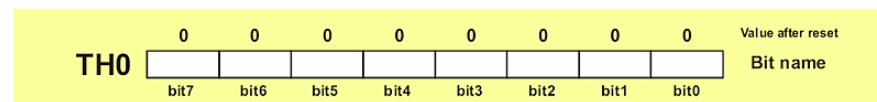
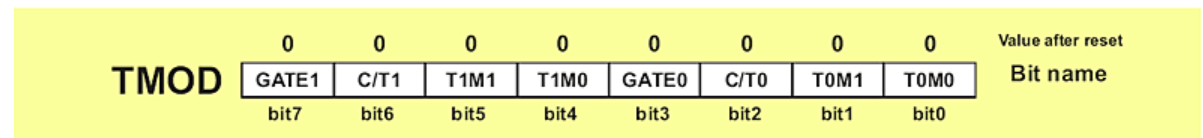
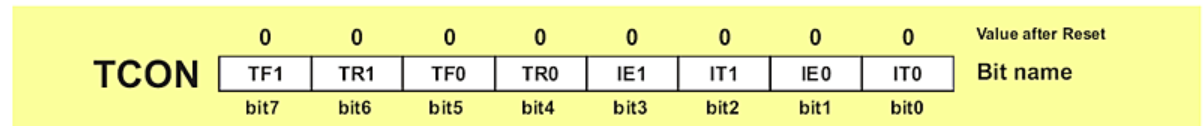
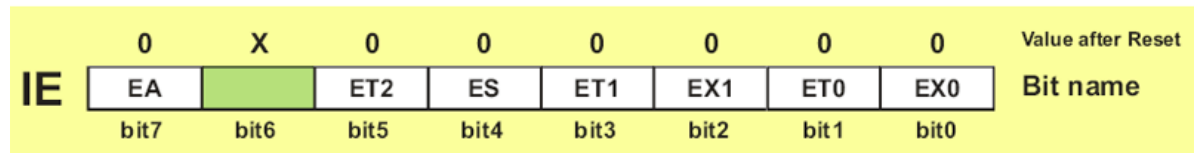


Exercise: Setup Timer Control Registers



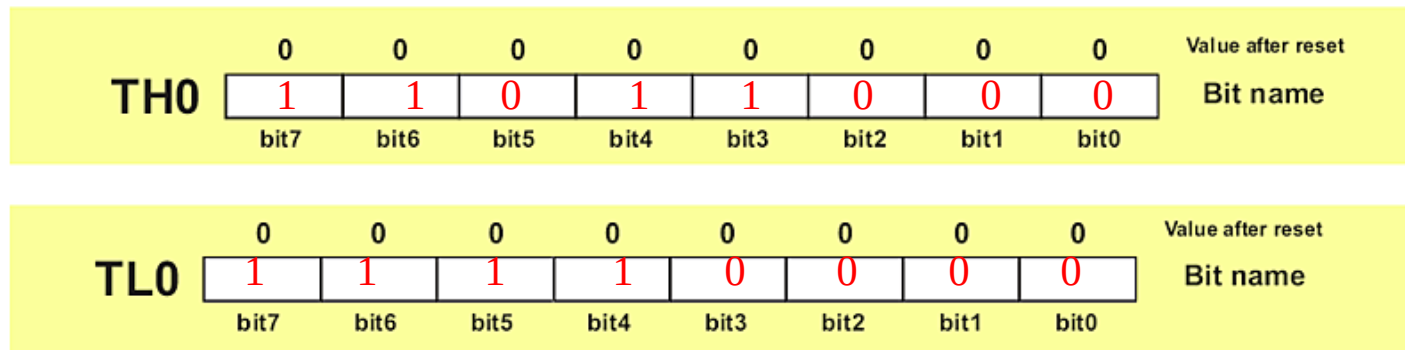
In-Class Exercise

- Suppose:
 - One cycle period of the timer counter is 0.1 ms
- Q: How to program 8051 to send an interrupt every 1 second?



The SFR Setup

- Setup the counter
 - Assuming it counts once every 0.1 ms
 - Count 10000 times for 1 second
 - $\{TH0, TL0\} = 65536 - 10000 = 55536 = (1101100011110000)_2$

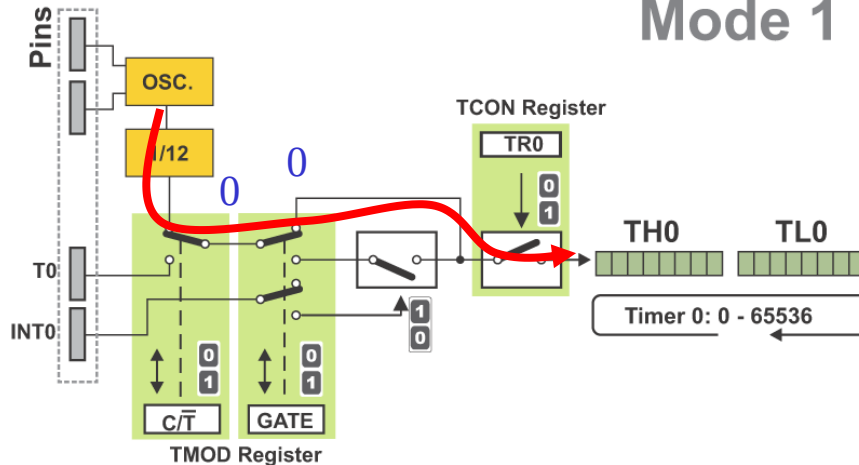


The SFR Setup

- Use timer 0 with mode 01
 - Mode 01: 16-bit timer

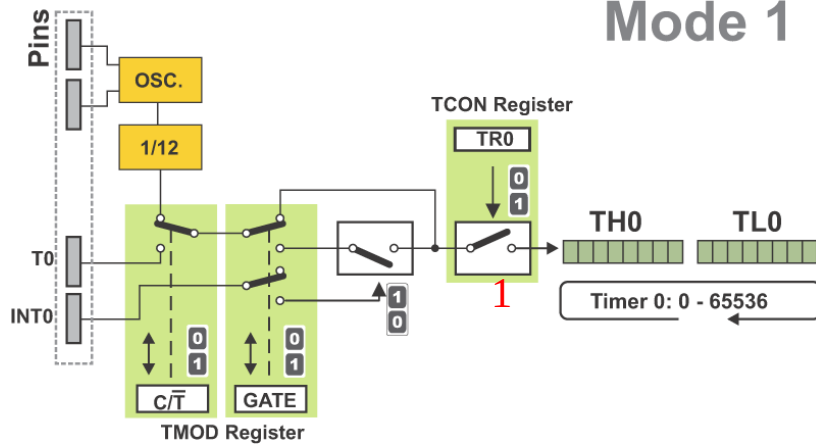
								Value after reset
TMOD	0	0	0	0	0	0	0	
	GATE1	C/T1	T1M1	T1M0	GATE0	C/T0	T0M1	T0M0
	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
				0	0	0	1	

Mode 1



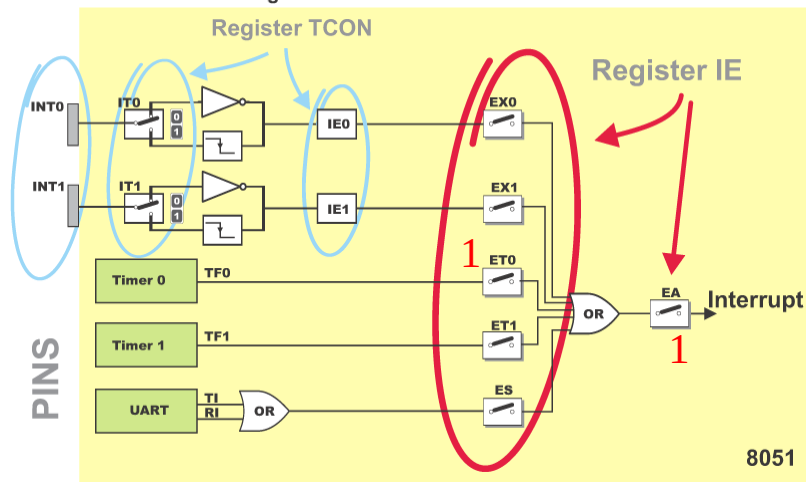
The SFR Setup

Mode 1



Value after Reset							
0	0	0	0	0	0	0	0
TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0

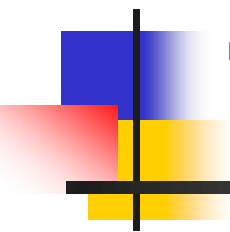
Bit name



Value after Reset							
0	X	0	0	0	0	0	0
EA		ET2	ES	ET1	EX1	ET0	EX0
bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0

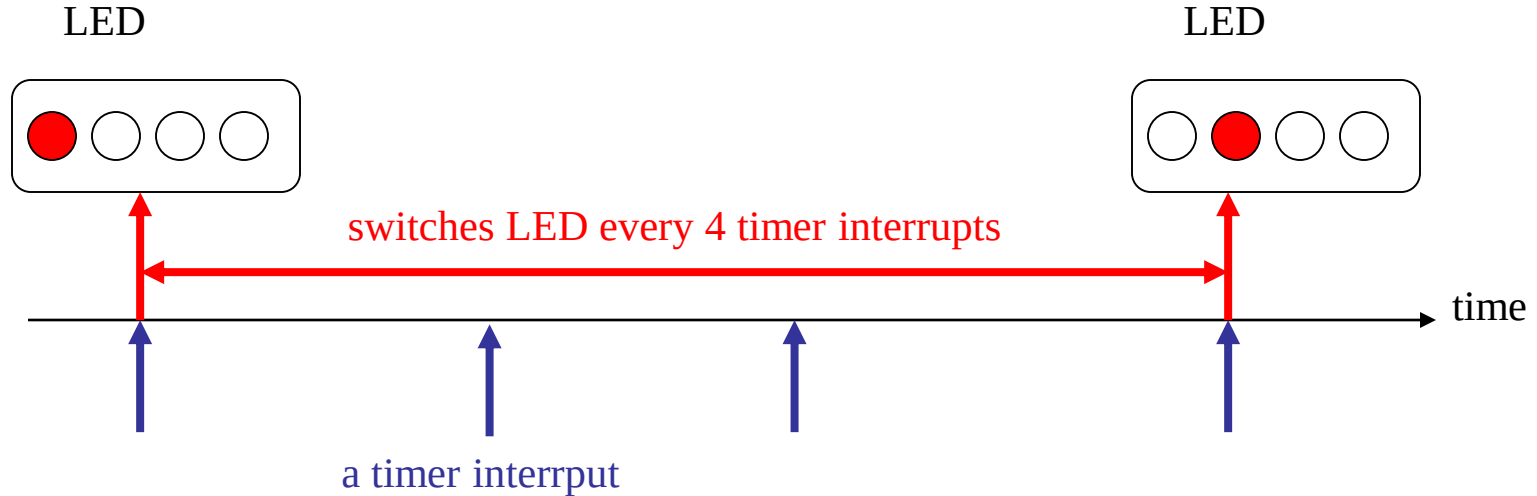
Bit name

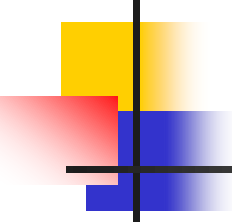
Demo: Make LED Run Using the Timer Interrupt



Function of the Demo

- Switches LED every 4 timer interrupts





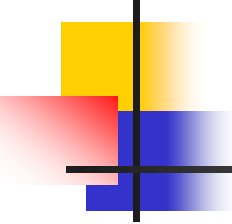
The Demo Program

- org to force program address
- And jump to actual ISR immediately

```
org    0h
ljmp   main

org    0bh
ljmp   Timer0_ISR

org    0100h
main:
ljmp   Timer0_ISR
lcall  Timer_Config
mov    R0, #4           ;the ISR entrance count
mov    R1, #80h         ;the LED pattern to display
loop:
mov    P2, R1
sjmp   loop
```



The Demo Program

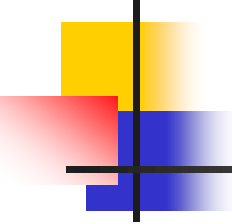
- Infinite loop to send control signals to LEDs

```
        org     0h
        ljmp    main

-----
        org     0bh
        ljmp    Timer0_ISR

main:
        lcall    Timer0_ISR
        lcall    Timer_Config
        mov     R0, #4           ;the ISR entrance count
        mov     R1, #80h        ;the LED pattern to display

loop:
        mov     P2, R1
        sjmp    loop
```



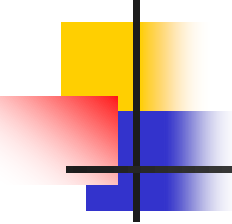
The Demo Program

- The timer ISR
- Change LED pattern every 4 times the ISR is executed

```
Timer0_ISR:
    DJNZ     R0, reset_timer

    mov     R0, #4
    mov     A, R1
    RL      A
    mov     R1, A

reset_timer:
    mov     TL0, #0
    mov     TH0, #0
    reti
    end
```

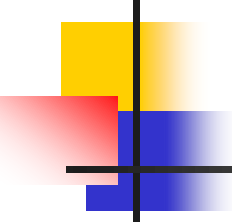


The Demo Program

- Setup the timer interrupt

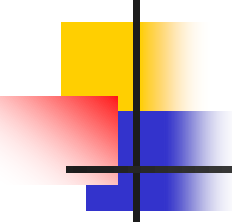
```
Timer_Config:
```

```
    mov     TMOD, #01h  
    mov     TCON, #010h  
    mov     TH0, #0  
    mov     IE, #082h  
    mov     TL0, #0  
    mov     TH0, #0  
    ret
```



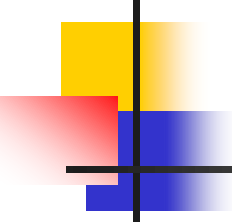
ONE PIECE

- 我也認真看過學長姊的版本了，其實很多都不是正確的，不論是新的板子還是舊的板子，大家都亂寫.....
- 範例程式中用：
(4次timer interrupts) x (65536次計數)
來代表一秒，其實是不精準的。



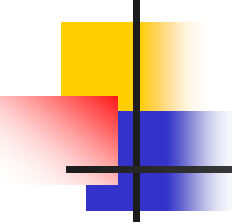
ONE PIECE

- 所以各位加分的寶藏就在那裡了！只要在**預報**中寫道如何正確地做出一秒鐘的延遲，且實驗中展現出來，加**10**分。
- 不要問我怎麼做，這是加分題。



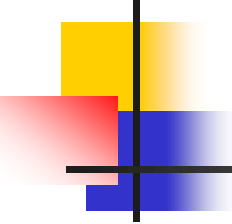
ONE PIECE

- 給點提示：
 - 我的算式：
$$65536 - (24.5\text{Mhz} / 8 / 12 / \text{跑4次}) = 1734$$
每次timer reset不該設成0，應是1734 (Dec.)
 - 關鍵字：System Clock, SYSCLK, OSCICN, CKCON
 - 從舊板子的手冊可以查到以上相關資訊：
<https://www.silabs.com/documents/public/data-sheets/C8051F04x.pdf>
 - 看電子書，看技術手冊絕對有幫助
 - <https://www.mikroe.com/ebooks/architecture-and-programming-of-8051-mcus>



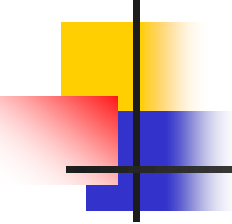
ONE PIECE

- 新的板子的算法不一樣：
 - https://www.nuvoton.com/export/resource-files/W78E054D_W78E052D_A13.pdf
 - 自己讀一下技術手冊絕對有幫助
 - 找找實驗板子上的相關資訊
 - 別亂掰答案，亂掰的沒有加分



Lab03 Study Report

- File name: Bxxxxxxx-MCE-Lab3-Study
- File type: PDF only
- The requirements of report
 - Summarize the content of this slide set
 - Provide your plan for this lab exercise
 - No more than one A4 page
 - Grading: 80 ± 15
- Deadline: 2026/10/21 23:00 (不收遲交)
- Upload to e-learning system



Lab03 Lab Exercise Report

- File name: Bxxxxxxx-MCE-Lab3-Result
- File type: PDF only
- The requirements of report
 - Summarize the problems and results you have in this exercise
 - Some screen shots or some code explanation can be provided
 - No more than two A4 pages
 - Grading: 80 ± 15
- Deadline: 2026/10/28 23:00 (不收遲交)
- Upload to e-learning system